

Lab 15: Course Review, Emerging Technologies & Career Paths

Z5008 Big Data Lab — Comprehensive Capstone Session

Dr. Innocent Nyalala
`innocent@iitmz.ac.in`

School of Engineering & Science
IIT Madras Zanzibar

Monday, 22 June 2026

Session Roadmap

Today's Agenda

- 1 **Course Visual Summary** (Labs 1–14)
- 2 **Emerging Technologies** (2025–2026)
- 3 **Career Paths in Big Data**
- 4 **Interview Preparation**
- 5 **Final Project Guidelines**
- 6 **Open Q&A & Retrospective**

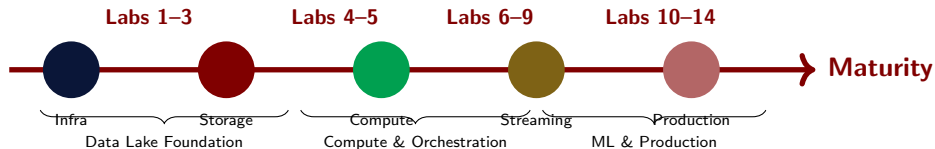
Key Dates

Today	Lab 15, 22 Jun 2026
Fri 26 Jun	Final project due 23:59 EAT
Submission	GitHub + Moodle

Final Deliverable

100 marks | 4 parts
Pipeline + Report + Slides + Reflection

The Learning Arc: From Containers to Production



Labs 1–4: Infrastructure, Storage & Compute Foundations

Infrastructure

Docker Compose, MinIO (S3-compatible), S3A connector, Hadoop filesystem abstraction. *Key skill:* spin up a data lake from scratch.

Apache Iceberg

Hidden partitioning, partition evolution, schema evolution, snapshot isolation, incremental reads. *Key skill:* table format trade-offs.

Delta Lake

ACID transactions, time travel, MERGE (upsert), medallion architecture (Bronze/Silver/Gold). *Key skill:* reliable mutable data lake tables.

Apache Spark

DAG execution model, lazy evaluation, RDDs vs DataFrames, shuffle joins, broadcast joins, persist/cache. *Key skill:* Spark job optimisation.

Labs 5–8: SQL, Orchestration & Streaming

Spark SQL & Optimisation

Window functions (ROW_NUMBER, LAG, LEAD), AQE, skew handling, bucketing, Z-ordering. *Key skill:* query tuning at scale.

Apache Kafka

Producers, consumers, consumer groups, partitioning strategy, Kafka Connect (source/sink), exactly-once semantics. *Key skill:* real-time event streaming.

Apache Airflow

DAG anatomy, BashOperator, PythonOperator, sensors, XComs, backfilling, SLA alerts. *Key skill:* production workflow scheduling.

Spark Structured Streaming

Micro-batch vs continuous, watermarks, event-time processing, streaming Delta Lake writes, checkpointing. *Key skill:* streaming pipelines end-to-end.

Labs 9–12: Visualisation, ML & MLOps

Visualisation

Plotly interactive charts, Dash web dashboards, Grafana + Prometheus metrics, real-time KPI panels. *Key skill:* data products for stakeholders.

MLflow

Experiment tracking, run comparison, artifact logging, model registry, staging/production transitions. *Key skill:* reproducible ML experiments.

Spark MLlib

ML Pipelines API, feature engineering (StringIndexer, VectorAssembler), Random Forest, GBT, cross-validation. *Key skill:* distributed ML at scale.

CI/CD & Kubernetes

Docker multi-stage builds, Helm charts, GitHub Actions pipelines, automated testing, rolling deployments. *Key skill:* DevOps for data pipelines.

Labs 13–14: Governance, Quality & Integration

Data Governance & Quality

Great Expectations suites, data contracts, GDPR compliance (right to erasure in Delta), Prometheus alerting, lineage. *Key skill*: trustworthy data pipelines.

Lab 13 Takeaway

Quality is not a post-processing step — it is engineered into the pipeline from Day 1.

End-to-End Integration

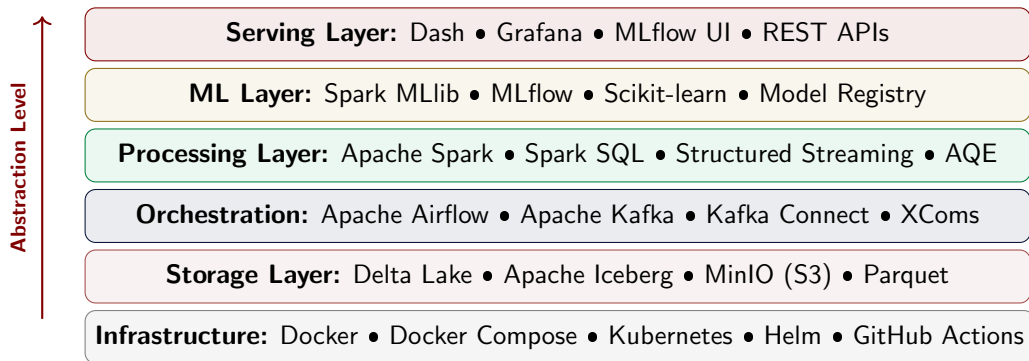
Lambda architecture (batch + speed layers), Kappa architecture (stream-only), full capstone pipeline, performance benchmarking. *Key skill*: system-level thinking.

Lambda vs Kappa

Lambda: separate batch & stream paths — consistency but operational complexity.

Kappa: single stream path — simpler but reprocessing is harder.

The Z5008 Technology Stack — Complete Picture



Emerging Technologies: What Is Next? (2025–2026)

The Landscape Evolves Fast

The tools you learned are production-grade today. But the ecosystem moves quickly. Understanding *why* new tools emerge prepares you to evaluate them critically as an engineer.

Three forces driving change:

- ➊ **Streaming lakehouse convergence** — batch and stream unified in one format
- ➋ **Open governance** — avoid vendor lock-in, open catalogs replacing Hive
- ➌ **AI integration** — LLMs embedded directly in data pipelines

Technologies We Survey

- Apache Paimon
- Project Nessie
- Apache Polaris
- DuckDB
- Apache Flink
- Ray
- Unity Catalog
- LLMs in Data Eng.
- Vector Databases
- Arrow Flight

Apache Paimon — The Streaming Lakehouse Table Format

What is Paimon?

Apache Paimon (formerly Flink Table Store) is a lakehouse table format designed for *high-frequency streaming writes* — where Delta Lake and Iceberg were optimised for batch-dominant workloads.

When to Choose Paimon?

- Sub-minute latency SLAs
- Continuous CDC ingestion (MySQL → Lake)
- High-frequency upserts

Key differentiators:

- **Changelog streams:** native CDC support
- **Lookup joins:** stream-table joins with low-latency lookups
- **Merge engines:** Deduplicate, Partial-Update, Aggregation
- **Native Flink + Spark support**

vs Delta Lake

Delta: excellent for batch + moderate streaming.

Paimon: excellent for continuous streaming upserts.

Both: ACID, time travel.

Project Nessie & Apache Polaris — Open Catalogs

Project Nessie

A **Git-like catalog** for Iceberg and Delta Lake tables.

- **Branches & tags** on your data
- Experiment on a branch; merge when ready
- Zero-copy branching (metadata only)
- Used by Dremio Arctic in production

Analogy: git checkout -b feature
but for your data warehouse.

Apache Polaris

Open-source Iceberg REST catalog, open-sourced by Snowflake in 2024; now an Apache incubating project.

- Vendor-neutral Iceberg catalog API
- Fine-grained access control
- Works with Spark, Flink, Trino
- Drop-in alternative to Hive Metastore

Why it matters: run your own catalog without cloud vendor dependency.

Catalog Landscape

What is DuckDB?

An in-process analytical database. No server required. Runs inside Python, R, Java. *Think SQLite, but column-oriented and OLAP-fast.*

Capabilities relevant to Z5008:

- Reads **Parquet**, **Delta Lake** (delta extension), **Iceberg** natively
- SQL over S3/MinIO with `httpfs` extension
- **Excellent for <1 TB** datasets on one node
- Parallelises across CPU cores automatically
- Embeds in Jupyter via `duckdb` Python package

DuckDB vs Spark

DuckDB	Spark
<1 TB	>1 TB
Single node	Cluster
Exploratory	Production ETL
Zero setup	Cluster mgmt

Ecosystem Fit

dbt + DuckDB = fast local

Flink vs Spark Structured Streaming

Both process streams but with different design philosophies.

	Flink	Spark SS
Execution model	True streaming	Micro-batch
Latency	Milliseconds	Seconds
State backend	RocksDB native	Checkpoints
Table format	Paimon, Kafka	Delta, Kafka
Learning curve	Steep	Moderate

Flink Strengths

- Fraud detection (ms latency)
- Complex event processing
- Large stateful operations (RocksDB)
- Exactly-once guarantees
- Deep Kafka integration

Career Relevance

Flink is widely used in fintech (M-Pesa, Stripe) and e-commerce (Alibaba is a major contributor).

Ray — Distributed Python for ML at Scale

What is Ray?

Ray extends Python across a cluster without Spark's JVM overhead. It is the foundation of most modern ML platforms.

Key components:

- **Ray Tune:** distributed hyperparameter optimisation
- **Ray Serve:** model serving with low latency
- **Ray Data:** parallel dataset processing
- **Ray Train:** distributed PyTorch / TensorFlow
- **Ray Core:** general task and actor parallelism

Ray vs Spark for ML

Ray	Spark MLlib
Python-native	Scala/Python
GPU-first	CPU-focused
Deep learning	Tabular ML
Anyscale SaaS	Databricks

Industry Adoption

OpenAI, Uber, Shopify, and Pinterest use Ray for ML training and serving at scale.

LLMs in the Data Stack & Vector Databases

LLMs in Data Engineering

Text-to-SQL:

DAIL-SQL, Vanna.ai — query your Delta Lake in plain English. Enterprise tools: Databricks DBRX, Google Duet AI.

LLM-based data quality:

Detect schema drift, classify anomalies using embeddings. Generate data quality rules from sample data.

Automated documentation:

Generate data dictionaries from column names + sample values. Summarise data lineage graphs in natural language.

Vector Databases

Store dense vector embeddings for similarity search. Core component of RAG (Retrieval-Augmented Generation) pipelines.

- **Pinecone:** managed, serverless
- **Weaviate:** open-source, hybrid search
- **pgvector:** vector extension for PostgreSQL
- **Chroma:** lightweight, developer-friendly

Data Engineering Use Case

Apache Arrow Flight & Databricks Unity Catalog

Apache Arrow Flight

High-speed bulk data transfer protocol built on gRPC with the Arrow columnar in-memory format.

- 10–100× faster than JDBC/ODBC for bulk reads
- Zero-copy column transfers between systems
- DuckDB, Spark, Pandas all speak Arrow natively
- *FlightSQL* extends it with SQL semantics

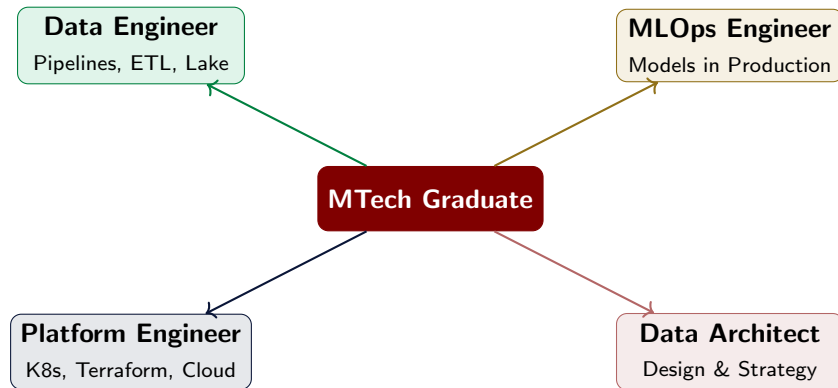
Significance: data movement is often the

Databricks Unity Catalog

Fine-grained governance layer for Delta Lake:

- Column-level and row-level security
- Automated end-to-end data lineage
- Attribute-based access control (ABAC)
- Audit logs for GDPR / HIPAA compliance
- Cross-workspace Delta Sharing

Career Paths in Big Data & Data Engineering



All four paths build on the same Z5008 technology foundation.

Career Path 1: Data Engineer

Core Responsibilities

- Design and maintain data ingestion pipelines
- Build and optimise lakehouse storage layers
- Ensure data quality and schema reliability
- Collaborate with data scientists and analysts
- Manage schema evolution and migrations

Tech Stack (2026)

Dr. Nyalala (IITMZ)

Salary Ranges (Annual, USD)

Region	Range
East Africa	\$18k–\$40k
South Africa	\$30k–\$65k
Europe	\$70k–\$110k
USA	\$110k–\$160k

Day-in-the-Life

Morning: triage Airflow DAG failures.

Midday: review dbt model pull requests.

Afternoon: design new ingestion pipeline.

End of day: check Great Expectations

dashboards

Career Path 2: **MLOps / ML Platform Engineer**

What MLOps Engineers Build

- **Feature stores:** Feast, Tecton, Hopsworks
- **Model serving:** Ray Serve, BentoML, Seldon
- **Training pipelines:** reproducible, versioned, distributed
- **Monitoring:** data drift (Evidently AI), model performance alerts
- **CI/CD for ML:** GitHub Actions + MLflow + automated tests

Career Progression

Junior MLE → MLOps Engineer →
Staff ML Ops → ML Platform Lead →

Key Certifications

- Databricks Certified ML Associate
- AWS ML Specialty
- Google Professional ML Engineer
- Coursera: MLOps Specialisation (Andrew Ng)

African Context

MLOps is emerging in Nairobi, Lagos, and Cape Town fintech and agritech sectors. SAAIL Lab research directly addresses this space.

Career Paths 3 & 4: Platform Engineer & Data Architect

Platform / DataOps Engineer

Focus on the infrastructure that *runs* data systems.

- Kubernetes cluster management
- Terraform / Pulumi infrastructure as code
- Cloud cost optimisation (spot instances, autoscaling)
- Security: RBAC, network policies, secrets management
- Observability: Prometheus, Grafana, OpenTelemetry

Often the highest-paying pure-infrastructure

Data Architect

Strategic design role:

- Define data platform strategy (cloud, tools, formats)
- System design for scalability and governance
- Create data mesh or data fabric architectures
- Evaluate build vs buy decisions
- Work closely with CTO and data leadership

Typically 8+ years experience required.

Classic Question: Design a Real-Time Fraud Detection System for M-Pesa

Structured Answer Framework (30 min):

- 1 **Clarify requirements** (5 min) — volume, latency SLA, false positive tolerance
- 2 **High-level architecture** — streaming + batch layers
- 3 **Component selection** — Kafka (ingestion), Flink/Spark SS (scoring), Delta Lake (history), MLflow (model versioning)
- 4 **Deep-dive one component** — e.g., feature engineering pipeline
- 5 **Scale & failure modes** — partitioning strategy, idempotency

Key Numbers to Know

M-Pesa TX/s	~1,000 peak
Fraud rate	~0.01–0.1%
Latency SLA	<500 ms
Look-back	30 days

Show Your Work

Always state your assumptions explicitly. Interviewers reward structured thinking over the single “right” answer.

Interview Preparation: Technical & Behavioural Questions

Technical Question Examples

Q: Explain Spark's shuffle and why it is expensive.

A: Shuffle = data movement across partitions/nodes. Causes: disk I/O, network transfer, serialisation. Triggered by: groupBy, sort-merge join, repartition. Optimisations: broadcast join for small tables, bucketing, AQE coalesce.

Q: Design a partitioning scheme for time-series M-Pesa data.

A: Partition by year/month/day. Sub-partition by region. Z-order on user_id + timestamp within partitions. Target 128 MB–1 GB partition files.

Behavioural: STAR Method

Q: Describe a time you optimised a slow Spark query.

S:ituation — 4-hour daily batch job.

T:ask — reduce runtime to <30 min.

A:ction — identified shuffle-heavy join; switched to broadcast join; added bucketing.

R:esult — 87% runtime reduction.

Recommended Certifications

Databricks Associate DE

Confluent Kafka Developer

Google Cloud Prof. DE

Open Source Contribution: Your Competitive Edge

Why Contribute?

- Signals deep technical competence to employers
- Learn production codebases used worldwide
- Build a public portfolio beyond academic work
- Network with maintainers and global engineers
- Some contributors get hired by sponsoring companies

How to Start (Step-by-Step)

- 1 Pick a project you *used* in this course
- 2 Read CONTRIBUTING.md thoroughly

Good First-Issue Projects

- **Apache Spark** — Python API docs
- **Apache Iceberg** — Python library tests
- **Apache Airflow** — provider operators
- **delta-rs** — Rust/Python Delta lib
- **Great Expectations** — new expectations
- **MLflow** — UI improvements, docs



Final Project: Structure & Requirements (100 Marks)

Part A — Complete Pipeline (40 marks)

- Full end-to-end: ingestion → storage → processing → ML → serving
- GitHub repo with README, architecture diagram, setup instructions
- All notebooks executed (outputs visible)
- Measurable performance benchmark

Part B — Technical Report (20 marks)

1000–1500 words covering:

- Architecture decisions & rationale
- Challenges faced and solutions
- Performance benchmarks with numbers
- What you would do differently

Part C — Presentation (20 marks)

- 10-min slide deck (≥ 12 slides)
- Cover: problem, architecture, demo, results, lessons learned

Part D — Reflection (20 marks)

500 words:

- 5 most impactful concepts (one per chosen lab)

How to Deliver a Strong 10-Minute Technical Presentation

Recommended Slide Structure (≥ 12 slides)

- 1 **Title & team** (1 slide)
- 2 **Problem statement** — why it matters (1–2)
- 3 **Data description** — source, volume, schema (1)
- 4 **Architecture diagram** — full pipeline (1)
- 5 **Implementation highlights** — key decisions (2–3)
- 6 **Live demo or screenshots** (1–2)
- 7 **Results & benchmarks** — actual numbers (1)

Presentation Tips

- Pre-run all cells; never live-code under pressure
- Have screenshots as a fallback if Docker is slow
- One main idea per slide — no walls of text
- Use real numbers: “5.2M records/38 s” $\gg$ “fast”
- Anticipate: “Why not X instead of Y?”

Common Mistakes

Course Retrospective: What We Built Together

Z5008 By the Numbers

Labs completed	15
Technologies mastered	20+
Docker services orchestrated	10+
Approximate lines of Spark code	~3,000+
End-to-end pipelines built	14+
ML models trained & tracked	5+
Weeks of MTech effort	15

The Mental Model Shift

You no longer think in rows and tables.

You think in **streams**, **partitions**, **lineage**,

contracts, and **systems**.

What to Do This Week

- 1 Complete and submit your final project
(due **Fri 26 Jun 23:59 EAT**)
- 2 Share your GitHub portfolio on LinkedIn
- 3 Star the projects you used:
Spark, Iceberg, Airflow
- 4 Find one good-first-issue to explore over the break

Stay in Touch

innocent@iitmz.ac.in

Questions?

Contact Dr. Nyalala

Email: innocent@iitmz.ac.in

Office: Room 105, Engineering Bldg

Office Hours: Wednesday 14:00–16:00
EAT

Lab: SAAIL Lab, IIT Madras Zanzibar

Final Project Deadline

Friday, 26 June 2026

23:59 EAT

Submit via Moodle + GitHub repo link.

Good luck — you are ready for this!

“Big data is not about the data. It is about the insight you extract from it at scale.”